

Final Project: BeautifulSoup Documentation Analytics System

1. Project Scenario

Your team works as a **Junior Data Engineering Team** in a software training company. The company wants to build a Python-based system that analyzes online technical documentation. The first target website is: BeautifulSoup Official Documentation (<https://www.crummy.com/software/BeautifulSoup/bs4/doc/>)

Your system must collect, clean, analyze, and report information from this documentation page using Python.

2. Main Project Objectives

Build a Python application that can:

1. Download the BeautifulSoup documentation page.
2. Parse the HTML content.
3. Extract useful structured data.
4. Store extracted data into CSV files.
5. Analyze the documentation using Pandas and NumPy.
6. Generate a final analytical report.

3. Required Features Feature

3.1. Feature 1: Web Page Collector

The system must use: requests to retrieve HTML content from the target URL.

Requirements

The program must:

- ✓ Send HTTP request to the target URL.
- ✓ Check HTTP status code.
- ✓ Save raw HTML into: data/raw/beautifulsoup_doc.html

3.2. Feature 2: HTML Parser

The system must use: **BeautifulSoup** to parse the downloaded HTML.

Notes: The official documentation explains that BeautifulSoup can work with Python's built-in `html.parser`, `lxml`, and `html5lib`, it also notes that `lxml` is faster while `html5lib` is more lenient but slower

3.3. Feature 3: Section Extractor

Extract all documentation sections.

Required Output

Create: `data/processed/sections.csv`

Required Columns: `section_id`, `section_level`, `section_title`, `section_text`, `word_count`, `code_block_count`, `link_count`

3.4. Feature 4: Link Extractor

Extract all hyperlinks from the documentation.

Notes: The official documentation demonstrates extracting URLs by iterating over `soup.find_all('a')` and reading each link's `href` attribute.

Required Output

Create: `data/processed/links.csv`

Required Columns: `link_text`, `href`, `link_type`, `section_title`

Link Type Rules

Classify each link as: `internal_anchor`, `external_link`, `documentation_link`, `image_link`, `empty_or_invalid`

3.5. Feature 5: Code Example Extractor

Extract all Python code examples from the documentation.

Required Output

Create: `data/processed/code_examples.csv`

Required Columns: `example_id`, `section_title`, `code_text`, `line_count`, `contains_find_all`, `contains_find`, `contains_select`, `contains_get_text`, `contains_requests`

3.6. Feature 6: Documentation Analytics

Using Pandas and NumPy, answer at least **8 analytical questions**.

Required Questions

1. How many sections are in the documentation?
2. Which section has the highest word count?
3. Which section contains the most code examples?
4. Which section contains the most links?
5. What are the top 10 most frequent technical keywords?
6. How many internal and external links exist?
7. How many code examples use `find_all()`?
8. How many code examples use `get_text()`?

Additional Questions

Each team must propose at least **2 additional analytical questions**.

3.7. Feature 7: Data Visualization

Create at least **4 charts**.

Required Charts:

- ✓ Bar chart: Top 10 sections by word count
- ✓ Bar chart: Number of code examples by section
- ✓ Pie chart: Link type distribution
- ✓ Histogram: Code example line count distribution

Charts must be saved in: `output/charts/`

3.8. Feature 8: Final Report Generator

- Create a final report including:
 - ✓ Dataset overview
 - ✓ Scraping method
 - ✓ Extracted data summary
 - ✓ Analysis results
 - ✓ Charts
 - ✓ Key findings
 - ✓ Limitations
 - ✓ Conclusion
- The final report can be submitted as: PDF or Jupyter Notebook exported to PDF

4. Suggested Project Structure

This is a sample final project structure for teams to refer to. Please adjust it to suit your team's needs.

File Edit View Run Kernel Settings Help

Python 3 (ipykernel)

FinalProject_BeautifulSoup_Analysis.ipynb

Filter files by name

FinalProject /

data

raw

beautifulsoup_doc.html

processed

sections.csv

links.csv

code_examples.csv

output

charts

word_count_by_section.png

code_examples_by_section.png

link_type_distribution.png

code_linecount_hist.png

final_report.pdf

summary_tables.xlsx

notebooks

FinalProject_BeautifulSoup_Analysis.ipynb

src

collector.py

parser.py

extractor.py

analyzer.py

visualizer.py

main.py

README.md

requirements.txt

Final Project – BeautifulSoup Documentation Analytics

Collect – Parse – Extract – Analyze – Visualize

In [1]:

Cell 1 (Markdown)

Tiêu đề và mô tả dự án

In [2]:

Cell 2 (Code)

Import các thư viện cần thiết

In [3]:

Cell 3 (Code)

Gửi request, lấy HTML và lưu file

In [4]:

Cell 4 (Code)

Parse HTML bằng BeautifulSoup

In [5]:

Cell 5 (Code)

Trích xuất các section (heading)

In [6]:

Cell 6 (Code)

Tạo DataFrame từ dữ liệu đã trích xuất và hiển thị

Out[6]:

	section_id	section_level	section_title	section_text	word_count	code_block_count	link_count
0	1	h1	Beautiful Soup Documentation	Beautiful Soup is a Python library...	120	3	5
1	2	h2	Getting Started	This chapter covers the basics...	98	2	3
2	3	h3	Installation	You can install BeautifulSoup...	45	1	2
3	3	h3	Basic Usage	Once installed, you can parse HTML...	76	2	4
4	2	h2	Navigating the Tree	Beautiful Soup represents documents...	210	4	6

OR:

FINALPROJECT

data

raw

beautifulsoup_doc.html

processed

sections.csv

links.csv

code_examples.csv

output

charts

word_count_by_section.png

code_examples_by_section.png

link_type_distribution.png

code_linecount_hist.png

final_report.pdf

summary_tables.xlsx

src

collector.py

parser.py

extractor.py

analyzer.py

visualizer.py

main.py

notebooks

analysis.ipynb

README.md

requirements.txt

FinalProject/

data/

raw/

beautifulsoup_doc.html

processed/

sections.csv

links.csv

code_examples.csv

output/

charts/

word_count_by_section.png

code_examples_by_section.png

link_type_distribution.png

code_linecount_hist.png

final_report.pdf

summary_tables.xlsx

src/

collector.py

parser.py

Extractor.py

analyzer.py

visualizer.py

main.py

notebooks/

analysis.ipynb

README.md

requirements.txt

All data files

Raw HTML downloaded from the website

Raw HTML file

Cleaned and extracted data

Extracted sections and their information

Extracted links

Extracted code examples

Outputs and reports

All charts generated from analysis

Bar chart: Top sections by word count

Bar chart: Code examples by section

Pie chart: Link type distribution

Histogram: Code example line count

Final analytical report

Summary tables used in the report

Source code

Download HTML using requests

Parse HTML and extract sections

Extract links and code examples

Data analysis with Pandas & NumPy

Create visualizations

Run the whole pipeline

Jupyter Notebook for analysis

Analysis workflow and exploration

Project overview and instructions

List of Python dependencies

5. Technical Requirements

Students must use:

- ✓ Python
 - ✓ Requests
 - ✓ BeautifulSoup4
 - ✓ Pandas
 - ✓ NumPy
 - ✓ Matplotlib
 - ✓ Jupyter Notebook
-

This is optional

Advanced Requirement: Local Analytics Application

Scenario:

After completing the data collection and analysis, the project manager requests your team to develop a local application so that non-technical users can use the BeautifulSoup Documentation Analytics System without opening Jupyter Notebook or modifying Python code.

The application must provide a graphical user interface (GUI) that allows users to execute the complete analytics workflow.

Option A: Desktop Application

Students may use one of the following frameworks:

- Tkinter (recommended)
- CustomTkinter
- PyQt5 / PySide6
- Kivy

The application runs locally on Windows.

Option B: Local Web Application

Instead of a desktop GUI, students may develop a local web application.

Recommended frameworks:

- Flask (recommended)
- Streamlit

- Dash
- FastAPI + HTML templates (optional)

The application runs on localhost.